

BUILD A CHATBOT USING HUGGING FACE TRANSFORMERS

AIM

To develop a simple conversational chatbot using the pre-trained Qwen2.5-1.5B-Instruct model from Hugging Face Transformers in Google Colab using CPU execution.

PLATFORM & REQUIREMENTS

Platform: Google Colab – CPU runtime. GPU is not required.

Requirements:

- Google Colab
- Internet connection for the initial model download
- Python 3.x (provided by Colab)
- PyTorch
- Hugging Face Transformers
- Accelerate

IMPORTANT INFORMATION

This program uses the Hugging Face Transformers library and runs the Qwen2.5-1.5B-Instruct model locally in the Colab runtime. It does not use the Hugging Face Inference API, OpenAI API, or any API key. CPU execution is supported, although response generation may be slower than GPU execution.

INSTRUCTIONS FOR STUDENTS

1. Open the provided Google Colab notebook.
2. Keep the runtime as CPU. GPU is not required.
3. Run the complete program cell.
4. Wait while the required libraries and model are downloaded.
5. Do not stop the cell while the model is loading.
6. When “Model loaded successfully!” appears, enter questions at the “You:” prompt.
7. Type “clear” to start a new conversation.
8. Type “exit”, “quit”, or “bye” to stop the chatbot.

COMPLETE PROGRAM

```
!pip install -q -U "transformers>=4.45,<5" "torch>=2.2" "accelerate>=0.30"

import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "Qwen/Qwen2.5-1.5B-Instruct"

print("Loading model...")
```

```

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float32
)
model = model.to("cpu")
model.eval()

print("Model loaded successfully!")
print("Device: CPU")

messages = [
    {
        "role": "system",
        "content": "You are a helpful AI assistant. Answer questions clearly and accurately."
    }
]

def chatbot(user_input):
    global messages
    messages.append({
        "role": "user",
        "content": user_input
    })

    text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    inputs = tokenizer(text, return_tensors="pt")

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=100,
            do_sample=True,
            temperature=0.7,
            top_p=0.9,
            pad_token_id=tokenizer.eos_token_id
        )

    response = tokenizer.decode(
        outputs[0][inputs["input_ids"].shape[-1]:],
        skip_special_tokens=True
    ).strip()

    messages.append({
        "role": "assistant",
        "content": response
    })

    return response

print("
" + "=" * 50)

```

```

print("QWEN AI CHATBOT")
print("=" * 50)
print("Running on CPU")
print("Type 'clear' to clear the conversation.")
print("Type 'exit' to stop.")
print("=" * 50)

while True:
    user_input = input("
You: ").strip()
    if not user_input:
        continue
    if user_input.lower() in ["exit", "quit", "bye"]:
        print("Bot: Goodbye!")
        break
    if user_input.lower() == "clear":
        messages = [
            {
                "role": "system",
                "content": "You are a helpful AI assistant. Answer questions
clearly and accurately."
            }
        ]
        print("Bot: Conversation cleared.")
        continue

    print("Bot:", chatbot(user_input))

```

SAMPLE INTERACTION

```

You: What is Artificial Intelligence?
Bot: Artificial Intelligence is a field of computer science that focuses on
creating systems capable of performing tasks that...

You: Explain machine learning.
Bot: Machine learning is a subset of artificial intelligence in which computers
learn patterns from data and use those pattern...

You: clear
Bot: Conversation cleared.

You: bye
Bot: Goodbye!

```

RESULT

Thus, a simple conversational AI chatbot was successfully developed using the pre-trained Qwen2.5-1.5B-Instruct model and Hugging Face Transformers in Google Colab using CPU execution. The chatbot accepts user queries, maintains conversation history, generates responses, and supports clear and exit commands without requiring an API key.

NOTE

The first execution requires an internet connection because the model and required libraries must be downloaded. CPU execution may take longer than GPU execution. Waiting during the initial model loading is normal. The model is intended for general conversational and instruction-following tasks; responses may not always be perfectly accurate.